



APE: FASTER AND LONGER CONTEXT-AUGMENTED GENERATION VIA ADAPTIVE PARALLEL ENCODING

Xinyu YangCarnegie Mellon University
xinyuagi@cmu.edu**Tianqi Chen**Carnegie Mellon University
tqchen@cmu.edu**Beidi Chen**Carnegie Mellon University
beidic@cmu.edu

ABSTRACT

Context-augmented generation (CAG) techniques, including RAG and ICL, require the efficient combination of multiple contexts to generate responses to user queries. Directly inputting these contexts sequentially introduce considerable computational burden by re-encoding the combined selection of contexts for every request. To address this, we explore the promising potential of parallel encoding to pre-compute and cache the KV states of each context independently. This approach enables direct loading of cached states during inference, while also accommodating more contexts through position reuse across contexts. However, due to misalignments in attention distribution, directly applying parallel encoding results in a significant performance drop. To enable effective and efficient CAG, we propose Adaptive Parallel Encoding (APE), which brings shared prefix, attention temperature, and scaling factor to align the distribution of parallel encoding with sequential encoding. Results on RAG and ICL tasks demonstrate that APE can preserve 98% and 93% sequential encoding performance using the same inputs, while outperforming parallel encoding by 3.6% and 7.9%, respectively. It also scales to many-shot CAG, effectively encoding hundreds of contexts in parallel. Efficiency evaluation demonstrates that APE achieves an end-to-end $4.5\times$ speedup by reducing $28\times$ prefilling time for a 128K-length context. The code is available at <https://github.com/Infini-AI-Lab/APE>.

1 INTRODUCTION

Recent advances in context-augmented generation (CAG), particularly retrieval-augmented generation (RAG) (Gupta et al., 2024; Gao et al., 2023) and in-context learning (ICL) (Dong et al., 2022; Wei et al., 2022), have been widely adopted in large language models (LLMs) (Dubey et al., 2024; Achiam et al., 2023), improving their ability to generalize to unseen tasks with contextual information, as demonstrated in Figure 1 (top). These techniques employ a *sequential encoding* process to ground LLM inputs with knowledge from external sources: concatenating the retrieved texts into one sequence, and encoding the sequence into key-value (KV) states as the context for subsequent queries. While this new, significantly longer input improves performance, the increased latency in context prefilling becomes a bottleneck in tasks that require long inputs but generate short outputs (Bai et al., 2023; Agarwal et al., 2024; Jiang et al., 2024b). For example, prefilling a 128K context can take 17 seconds, whereas generating 256 tokens requires only 6 seconds. This discrepancy leaves significant room to improve the practical efficiency of CAG systems when deployed in real-world applications (Liu, 2022; Chase, 2022).

Since these contexts are typically predetermined and stored independently in external databases (Zaryani et al., 2024; Douze et al., 2024), one might naturally accelerate CAG systems by pre-caching them for direct loading during inference. However, for autoregressive LLMs, the KV states are inherently context-dependent, which means that the KV states for the same text vary based on the preceding context. This dependency necessitates caching all possible context permutations, leading to factorial growth in memory requirements as the database size increases. For instance, caching all permutations of just ten 256-token texts for the LLAMA-3-8B model would demand an impractical 22 PB of memory.

To address this issue, *parallel encoding* (Ratner et al., 2022; Yen et al., 2024; Li et al., 2024; Sun et al., 2024) encodes each context into KV states separately, ensuring that tokens from different contexts cannot be attended in the encoding phase. Next, the on-the-fly generation is started by prefilling user queries, which can attend to the cached KV states from all contexts without re-encoding, offering two benefits:

Pre-caching Contexts for Fast Inference: Contexts from the external sources can be pre-computed and cached into KV states for direct loading during inference. Additionally, this approach allows for cost-free manipulation of contexts, including insertion, deletion, replacement, and swapping operations.

Re-using Positions for Long Context: Contexts can be inserted into the same range of positions in an LLM’s context window, allowing for more and longer context chunks. This also avoids the "positional bias" in context ordering (Liu et al., 2024a), as each context is equally "close" to the generated tokens.

Problem Setup: Context-Augmented Generation

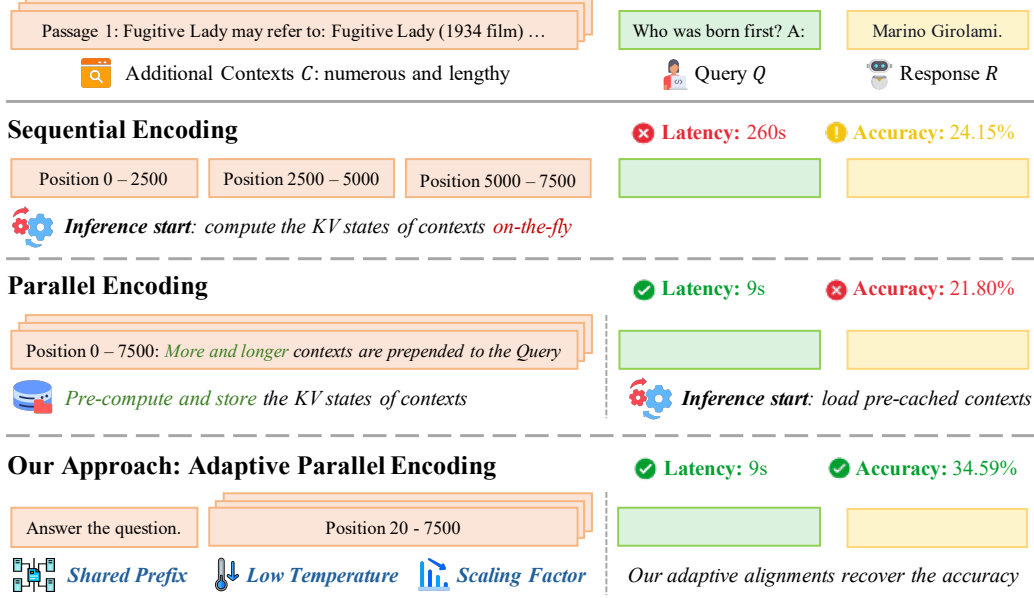


Figure 1: **Overview of our approach.** CAG uses additional contexts to improve the quality of LLM responses to queries. Sequential encoding prefills selected context chunks as a long sequence during inference, suffering from high latency due to on-the-fly re-encoding and low accuracy due to chunk size and count limits. Parallel encoding offers an alternative method to pre-cache more and longer contexts and share same positions across them but leads to worse performance. We propose Adaptive Parallel Encoding to re-align the attention weight distribution of parallel encoding with sequential encoding via three steps: shared prefix, scaling factor, and adaptive temperature. APE can accommodate more and longer contexts without re-encoding and fine-tuning, leading to fast and accurate CAG systems in practice.

Despite these advantages, parallel encoding leads to significant performance degradation across multiple scenarios as shown in Figure 2, with average declines of 4.9% (despite using 2-10 $\times$ more contexts) and 49.0%, respectively. While prior works (Sun et al., 2024; Yen et al., 2024) have attempted to correct this with fine-tuning, these methods continue to exhibit reduced accuracy in reasoning tasks. This performance drop can be attributed to the limited generalization ability from simple to complex tasks.

Additionally, our results in Figure 2 reveal that parallel encoding remains functional even without additional modifications, producing reasonable responses. This observation highlights the inherent alignments between sequential and parallel encoding. Our key insight from Figure 3 and 4 is that *KV states from independently encoded contexts can be naturally merged into one single sequence due to their similarity in both direction and magnitude, attributed to the presence of an attention sink* (Xiao et al., 2023). The sink ensures that KV states for distinct initial tokens exhibit a high cosine similarity and comparable L^2 norm. This initial alignment subsequently influences the direction of following KV states, which consistently maintain a similar large angle with the sink tokens (see Figure 5). To further understand the performance drop, we also identify and analyze the remaining misalignments.

Motivated by the above insights, we propose **Adaptive Parallel Encoding (APE)**, which aligns the distribution of attention weights between sequential encoding and parallel encoding. APE enables accurate and fast CAG, as demonstrated in Figure 1 (bottom). The main contribution of this paper is as follows:

- We systematically analyze the distribution properties of attention weights in parallel encoding, focusing on the magnitude and direction of KV states across various samples and positions. Our analyses identify both alignments and misalignments between parallel encoding and sequential encoding.
- We propose APE to recover the accuracy of parallel encoding with three alignment steps: (i) Prepend a shared prefix to avoid the duplication of abnormal initial token distributions. (ii) Adjust a lower attention temperature to sharpen the distribution, focusing on important tokens. (iii) Apply a scaling factor to offset the increase in the magnitude of the LogSumExp value of attention scores from the context.
- We empirically show that (i) APE keeps 98% and 93% of the sequential encoding performance in RAG and ICL tasks, respectively. (ii) APE outperforms parallel encoding in RAG and ICL, yielding improvements of 3.6% and 7.9%, respectively. (iii) APE effectively scales to handle hundreds of contexts, approaching or even surpassing sequential encoding in many-shot scenarios. (iv) APE accelerates long-context generation, achieving up to a 4.5 $\times$ speedup through 28 $\times$ reduction in prefilling time.

2 BACKGROUND AND RELATED WORK

2.1 CONTEXT-AUGMENTED GENERATION

In this work, we explore CAG with LLMs, where user queries are enhanced with additional contexts from external databases. CAG typically involves two scenarios: RAG (Asai et al., 2024; Gupta et al., 2024; Gao et al., 2023), which focuses on directly retrieving relevant information, and ICL (Dong et al., 2022; Wei et al., 2022), which emphasizes the transfer of capabilities by learning from examples.

2.2 PARALLEL ENCODING

Next, we present the formulation of using parallel encoding in LLMs for CAG. Let $\mathcal{S}$ represent the input sequence including N contexts $C_1, \dots, C_N$ and one query Q . Formally, this can be denoted as:

$$\mathcal{S} = \underbrace{\{s_{C_1,1}, \dots, s_{C_1,l_1}\}}_{\text{Context 1}}, \underbrace{\{s_{C_2,1}, \dots, s_{C_2,l_2}\}}_{\text{Context 2}}, \dots, \underbrace{\{s_{C_N,1}, \dots, s_{C_N,l_N}\}}_{\text{Context N}}, \underbrace{\{s_{Q,1}, \dots, s_{Q,l_Q}\}}_{\text{Query}}. \quad (1)$$

For simplicity, we can express this as: $\mathcal{S} = \{S_{C_1}, S_{C_2}, \dots, S_{C_N}, S_Q\}$. Given two models Θ_{Enc} and Θ_{Dec} (which may be identical), a response $\mathcal{R}$ is generated to input $\mathcal{S}$ using parallel encoding through two steps:

Pre-caching Contexts. The initial step involves encoding and caching the KV states for each context independently using Θ_{Enc} . For a given context S_{C_i} , we compute the KV states as $(K_{C_i}, V_{C_i}) = \Theta_{\text{Enc}}(S_{C_i})$ and store them for subsequent direct use, where $K_{C_i} = \{k_{C_i,1}, \dots, k_{C_i,l_i}\}$ and $V_{C_i} = \{v_{C_i,1}, \dots, v_{C_i,l_i}\}$.

Generating Response. Next, we input the query and load all relevant KV states to generate response: $\mathcal{R} = \Theta_{\text{Dec}}(S_Q, K_C, V_C)$, where K_C, V_C are subsets of $\{K_{C_1}, \dots, K_{C_N}\}$ and $\{V_{C_1}, \dots, V_{C_N}\}$, respectively.

Parallel encoding significantly improves efficiency compared to sequential encoding by reducing complexity from $O((l_1 + \dots + l_N + l_Q)^2)$ (i.e., quadratic) to linear with respect to context length. With pre-caching, the cost becomes $O((l_1 + \dots + l_N + l_Q) \cdot l_Q)$, and without pre-caching, the $O(\max(l_1^2, \dots, l_N^2) + (l_1 + \dots + l_N + l_Q) \cdot l_Q)$ complexity is well-suited for cases involving multiple similar-length contexts.

Previous parallel encoding approaches vary in their implementation of Θ_{Enc} and Θ_{Dec} . Parallel Context Windows (PCW) (Ratner et al., 2022) directly employ the pre-trained LLM as both, resulting in significant performance drops. Block-Attention (Sun et al., 2024) further fine-tuning the model, successfully recovering performance in RAG tasks. Alternatively, CEPE (Yen et al., 2024) and FocusLLM (Li et al., 2024) train additional transformers for Θ_{Enc} , using encoder-only and decoder-only architectures respectively. These methods also differ in their design of Θ_{Dec} : CEPE train additional cross-attention layers for processing contexts, whereas FocusLLM directly input the context into the original self-attention layers. While these trainable approaches show promising results in RAG tasks, challenges remain regarding their training costs and generalization abilities to more complex ICL scenarios.

2.3 ATTENTION MECHANISM

In a standard Softmax Attention, we attend the query to all KV states using the following formula:

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad Q \in \mathbb{R}^{n \times d} \quad K, V \in \mathbb{R}^{m \times d}, \quad (2)$$

where Q is the query state, and K and V denote the key and value states, respectively.

Previous research has revealed several significant insights into the distribution of attention weights (i.e., $\text{Softmax}(\frac{QK^T}{\sqrt{d}})$), which has motivated the design of our Adaptive Parallel Encoding method.

Attention Sink. StreamingLLM (Xiao et al., 2023) identifies the presence of an ‘‘attention sink’’ in LLMs – a token that receives a significantly higher attention score than other tokens but provides limited semantic information. It observes that the attention sink only exists in the initial token.

Position Embedding. To effectively process sequential input, LLMs necessitate the explicit encoding of positional information. Common techniques include absolute positional embedding Vaswani (2017); Devlin (2018) and relative positional encoding (Su et al., 2024; Press et al., 2021). However, the introduction of position embedding not only limits the context window to the training length (Chen et al., 2023), but also results in the ‘‘lost in the middle’’ phenomenon (Liu et al., 2024a), where LLMs struggle to produce correct answers when relevant information is located in the middle of the context.

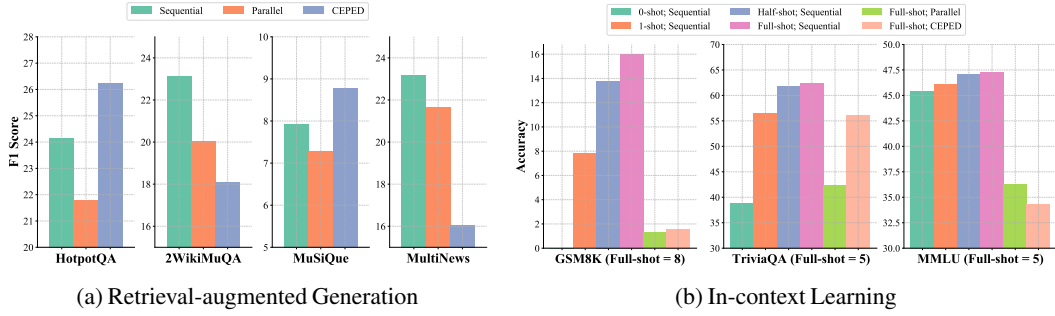


Figure 2: Comparison of sequential encoding, parallel encoding, and CEPED in RAG and ICL scenarios. Parallel encoding and CEPED dramatically degrades performance, especially for the math reasoning.

3 OBSERVATIONS

In this section, we evaluate sequential encoding, parallel encoding, and CEPE-Distilled (CEPED) (Yen et al., 2024) using the LLAMA-2-7B-CHAT model.¹ on various RAG and ICL tasks in Figure 2. Our analysis yields several key observations of parallel encoding. In Section 3.1, we explain the limitation of trainable approaches in their generalization abilities to complex tasks. Then, we explore why parallel encoding does not completely break down, despite LLMs being trained sequentially in Section 3.2. This analysis illuminates both alignments and misalignments between parallel and sequential encoding.

3.1 TRAINABLE APPROACHES ARE ONLY EFFECTIVE FOR EASY TASKS.

In Figure 2, we compare the performance of different context encoding methods on RAG and ICL tasks, with detailed setups described in Appendix A. Our analysis of the long-context RAG ability on LongBench (Bai et al., 2023) is showcased in Figure 2a. Despite accessing larger context windows, CEPED only surpasses the sequential baseline in 2/3 QA tasks, and it even notably underperforms parallel encoding in the Summarization task (MultiNews), which requires synthesizing information from the entire context. We hypothesize that CEPED cannot process complex tasks since the encoder and decoder are only trained unlabeled pre-training corpus, without instruction-tuning on high-quality QA samples. This is further supported by the results of ICL tasks (see Figure 2b), where CEPED performs on par with 1-shot sequential encoding baseline on TriviaQA but falls short of it on GSM8K and MMLU, despite using much more examples, respectively. The latter involves reasoning steps hard for the ill-trained model to understand. In conclusion, fine-tuning models to improve parallel encoding requires (i) more diverse and labeled data, and (ii) resource-intensive instruction-tuning (e.g., SFT or RLHF (Ouyang et al., 2022)). Given this unfavorable trade-off between training costs and model capabilities, we propose developing a training-free solution for better parallel encoding performance.

3.2 COMPARING PARALLEL ENCODING AND SEQUENTIAL ENCODING.

In Figure 2, we surprisingly observe that parallel encoding, despite reducing performance, can generate reasonable responses without further modifications. This is non-trivial as contexts are encoded into KV states separately, with no guarantee that these states can be compared or combined. However, our analysis reveals that the attention mechanism in LLMs naturally builds an inherent alignment across independent contexts similar to sequential encoding, despite the change in context and position. To clarify this, Figure 3 focuses on the impact of the attention sink (Xiao et al., 2023), where we visualize the direction (cosine similarity) of KV states for different samples and positions. In Figure 4, we further visualize the distribution of different components in the Softmax attention, resulting in several findings.

Key states from different contexts are similar. In Figure 3a and 3b, we measure the cosine similarity of the key states for different initial tokens for the LLAMA-3-8B-INSTRUCT and MISTRAL-7B-INSTRUCT-V0.3 models, which consistently yields a value close to 1. This indicates that the direction of the initial key state remains largely invariant across different inputs. Figure 3c and 3d further analyze the similarity between the initial key states and their subsequent states, where we observe comparable negative values from different positions. Therefore, the angles between the initial key states and their subsequent states are similar and significantly larger than the angles between different initial key states, as visualized in Figure 5. This suggests that the direction of key states remains relatively consistent across contexts, as they are primarily decided by the initial key states, which exhibit similar directions across examples. These findings, combined with the small variance in magnitude across examples in Figure 4b, show that key states from different contexts share similar directions and magnitudes, making them comparable.

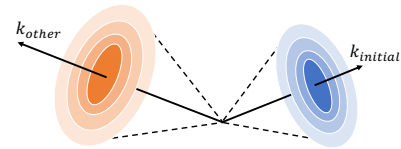


Figure 5: Geometry of Key States.

¹We use LLAMA-2 for CEPED, as it is the only supported model. For other analyses, we employ LLAMA-3.

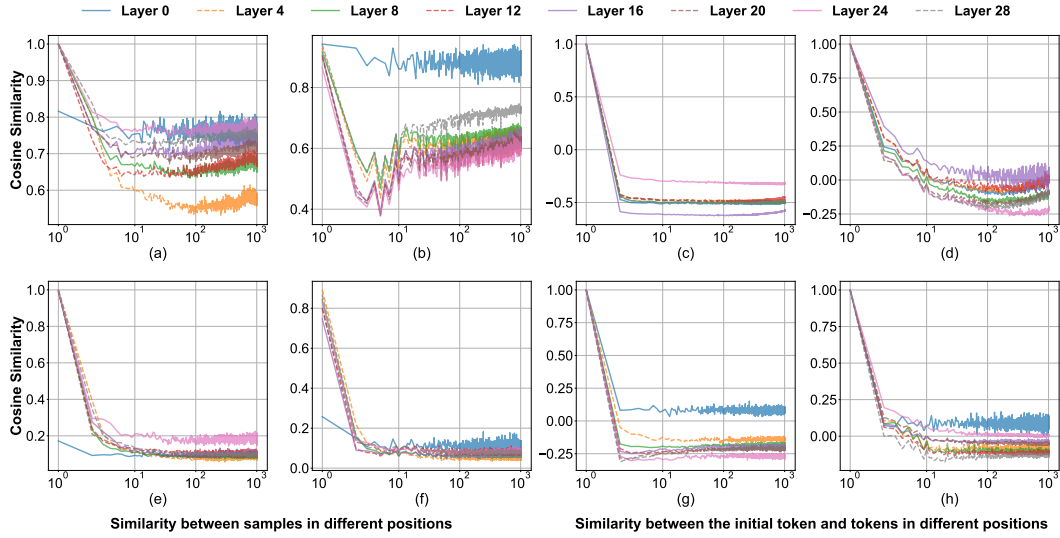


Figure 3: **Top Left:** Both LLAMA-3-8B-INSTRUCT (a) and MISTRAL-7B-INSTRUCT-v0.3 (b) states exhibit a cosine similarity larger than 0.9 for the key states from distinct initial tokens. **Top Right:** Initial token’s key states show similar negative values to those from other positions for LLAMA-3-8B-INSTRUCT (c) and MISTRAL-7B-INSTRUCT-v0.3 (d) models. **Bottom:** Value states exhibit patterns similar to those observed in key states. The X-axis shows positions of key states on a logarithmic scale.

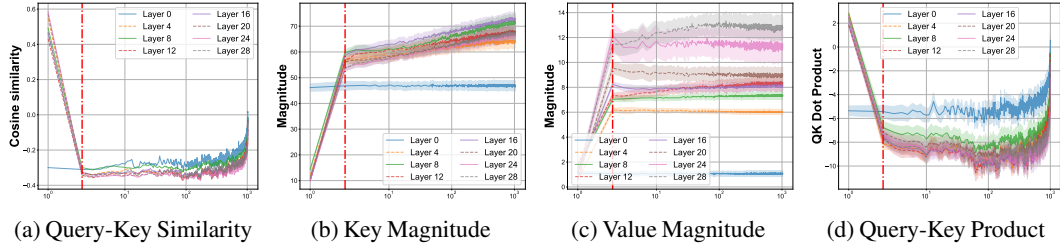


Figure 4: (a) The cosine similarity between query and key states increases as the distance between their positions decreases. (b) The magnitudes of key states show a slowly upward trend as position increases. (c) The magnitude of value states remain constant across positions. (d) Query-key dot products keep consistently low values except at initial and recent positions. A red dashed line marks the anomalous region for the first two tokens in all subfigures. The X-axis shows positions of key and value states on a log scale. Results are measured on the Hotpot QA dataset using the LLAMA-3-8B-INSTRUCT model.

To further understand this key observation, we conducted a small experiment using the LLAMA-3-8B-INSTRUCT model on the HotPotQA dataset. Our analysis involved applying rotations of varying degrees around random axes to the key states of initial tokens. For parallel encoding, we explored two rotation modes: one using the same rotation axis across different contexts, and another employing random rotation axis for each context. Figure 6 presents our findings, revealing that sequential encoding maintains performance across various rotation degrees. In contrast, both modes in parallel encoding deteriorate when rotations exceed 150 degrees. This effect arises from the duplication of initial tokens’ key states, which intensifies the impact of our rotations. Notably, using separate axes for each context leads to an earlier breakdown beginning at 90 degrees. This disrupts the directional similarity of key states with different initial tokens (i.e., k_{initial}) in Figure 5 and enlarges the angle between key states from different contexts.

Values states from different contexts can be combined. In equation 2, value states are combined through a weighted summation in the attention calculation, where the Softmax operator would normalize the weights (i.e., attention scores) of all value states to sum to 1. This normalization indicates that the magnitude of current value states is determined solely by those from previous positions,

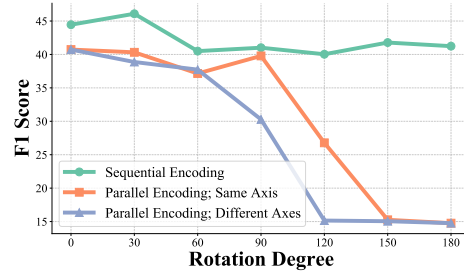


Figure 6: Rotation Analysis

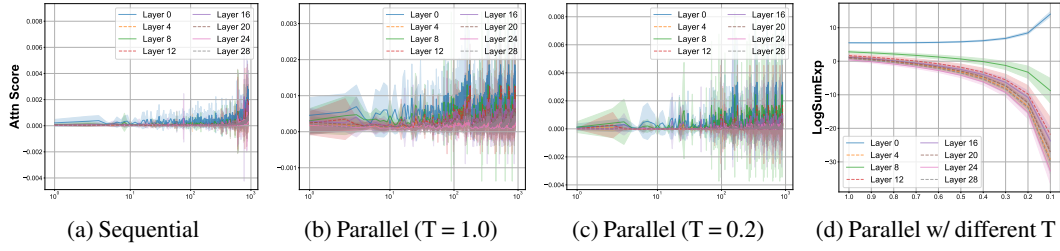


Figure 7: **(a)** Sequential encoding allocates a high attention score to neighboring tokens. **(b)** Parallel encoding distributes attention scores more uniform across more neighboring tokens. **(c)** Adjusting the temperature T sparsifies the distribution. **(d)** The total attention score allocated to contexts varies with the decrease in T . The X-axis in (a)-(c) represents token positions on a logarithmic scale, while in (d), it represents the value of T . Results are measured on the Hotpot QA using LLAMA-3-8B-INSTRUCT, resulting in a similar L^2 norm across positions, as shown in Figure 4c. Additionally, the small variance demonstrates that the magnitudes are comparable among samples. This, coupled with a similar direction across samples and positions in Figure 3 (Bottom), indicates that the value states can be combined.

Opportunities for improvement. Previous analyzes demonstrate that key and value states exhibit a natural alignment across contexts both in direction and magnitude for most positions. However, residual misalignments in Figure 4 still severely reduce performance. We summarize them as follows:

- In Figure 4, we observe a notable discrepancy in direction and magnitude for the first few positions, leading to large QK dot products in Figure 4d. This is designated as an abnormal region in the context.
- Figure 4d shows dot products between the final query state and all key states, revealing a notable increase when states are close to each other in position due to the larger cosine similarity in Figure 4a.

4 ADAPTIVE PARALLEL ENCODING

With all the lessons learned in Section 3, we will design our Adaptive Parallel Encoding (APE) to address the observed misalignments. APE enables a seamless shift to parallel encoding without requiring retraining, while maintaining the majority of the model’s capabilities. Our approach adaptively aligns the distribution of attention weights between sequential encoding and parallel encoding through a three-step procedure as showcased in Figure 1 (Bottom), thereby boosting both efficiency and accuracy.

4.1 PREPENDING SHARED PREFIX.

Figure 4 illustrates that the distribution of various components for the first few tokens differ significantly from those of subsequent tokens. This discrepancy poses a challenge when encoding contexts in parallel from the beginning, as it would result in the duplication of these abnormal KV states. To address this issue, we propose a simple yet effective solution: prepending a shared prefix to all contexts. This approach ensures that these KV states appear only once in each generation step. In practice, the choice of prefix varies with the model and task. We use existing system prompts and instructions as the shared prefix when available. Otherwise, we will insert a few newline characters (i.e., “\n”) before all contexts.

Additionally, as the magnitudes of key states increase approximately linearly on a logarithmic x-axis, their change is initially rapid and gradually slows down (similar for value states). Consequently, we also consider extending the existing prefix with additional newline characters to mitigate the discrepancy.

4.2 ADJUSTING ATTENTION TEMPERATURE.

In Figures 4a and 4d, the cosine similarities and corresponding dot products between the query and key states increase as their distance decreases, with a notably sharper rise when the distance approaches zero. To visualize its impact on parallel encoding, we set a 50-token prefix and query, encoding the remaining 900 tokens either sequentially or in five parallel segments. The resulting attention distributions are shown in Figure 7. Comparing Figure 7b with 7a, duplicating these neighboring KV states in parallel encoding will disperse query’s attention to multiple contexts, resulting in a more uniform distribution. To enhance focus on the most relevant tokens, we adjust the attention temperature T to a value less than 1, sharpening the distribution after exponentiation. An example with $T = 0.2$ is illustrated in Figure 7c.

4.3 ADDING SCALING FACTOR.

While adjusting the temperature sharpens the attention distribution among context tokens but also alters the overall attention allocated to them, as indicated by the LogSumExp value in Figure 7d. Specifically, when the sum of the original QK dot product values is significantly greater than 0, reducing T amplifies these positive values, resulting in an increased, positive LogSumExp value. Conversely, when the sum is closer to 0, lowering T has a stronger effect on the negative QK dot products, leading to a decreased, negative LogSumExp value. These effects generally increase the absolute value of LogSumExp. To compensate for these changes, we introduce a scaling factor $S < 1$ to directly reduce this absolute value.

4.4 FORMULATION.

Given these alignment processes, we can formulate the modified attention in APE. We begin with the standard Softmax attention, where Q , K , and V are the query, key, and value states from the input:

$$O = \text{Softmax}\left(\frac{Q[K_{C_1}^\top, \dots, K_{C_N}^\top, K_Q^\top]}{\sqrt{d}}\right) \times [V_{C_1}, \dots, V_{C_N}, V_Q] \quad (3)$$

$$= \frac{[A_{C_1}, \dots, A_{C_N}, A]}{\sum_{i=1}^N \sum_{j=1}^{l_{C_i}} a_{C_i,j} + \sum_{j=1}^{l_Q} a_{Q,j}} \times [V_{C_1}, \dots, V_{C_N}, V_Q], \quad (4)$$

where $A_{C_i} = [\exp \frac{Qk_{C_i,1}^\top}{\sqrt{d}}, \dots, \exp \frac{Qk_{C_i,l_{C_i}}^\top}{\sqrt{d}}]$ and $a_{C_i,j} = \exp \frac{Qk_{C_i,j}^\top}{\sqrt{d}}$. Similar for A and $a_{Q,j}$.

After incorporating the proposed changes, the formula for our refined attention calculation becomes:

$$O' = \frac{[A_P, A'_{C_1}, \dots, A'_{C_N}, A_Q]}{\sum_{j=1}^{l_P} a_{P,j} + (\sum_{i=1}^N \sum_{j=1}^{l_{C_i}} a'_{C_i,j})^S + \sum_{j=1}^{l_Q} a_{Q,j}} \times [V_P, V_{C_1}, \dots, V_{C_N}, V_Q], \quad (5)$$

$$\text{where } A'_{C_i} = [\exp \frac{Qk_{C_i,1}^\top}{T\sqrt{d}}, \dots, \exp \frac{Qk_{C_i,l_{C_i}}^\top}{T\sqrt{d}}] \cdot (\sum_{i=1}^N \sum_{j=1}^{l_{C_i}} a'_{C_i,j})^{S-1} \text{ and } a'_{C_i,j} = \exp \frac{Qk_{C_i,j}^\top}{T\sqrt{d}}.$$

Here, A_P represent the attention weights for the shared prefix, respectively. The scaling factor s and the attention temperature T for the context are less than 1 ($S < 1, T < 1$). All these modifications can be fused into fast attention implementations such as flash attention (Dao et al., 2022) with negligible cost.

For the choice of hyperparameters, we conduct a greedy search over the validation set. We begin by adding two "\n" if no prefix is provided, then increase the prefix length by 10, 20, and 40 for the best one. S and T are searched within the ranges $[0.1, 1.0]$ and $[0.1, 1.0]$ respectively, using 0.1 step sizes.

5 EXPERIMENTS

In this section, we present the effectiveness and efficiency of APE, focusing on CAG scenarios such as RAG and ICL. Therefore, we do not include comparisons with long-context LLMs here. Specifically,

- In Section 5.1, APE can maintain 98% of the accuracy on ChatRAG-Bench compared to sequential encoding when retrieving the same text as input. Furthermore, it improves performance by 3.3% on average for RAG on LongBench through its ability to retrieve and process more and longer contexts.
- In Section 5.2, APE outperforms parallel encoding by 7.9% on average in ICL tasks. Moreover, APE keeps 93% of accuracy achieved by sequential encoding when using the same number of examples.
- In Section 5.3, APE scales to many-shot CAG tasks, effectively encoding hundreds of texts in parallel.
- In Section 5.4, APE can achieve up to $4.5\times$ faster inference through $28\times$ reduction in prefilling time.

5.1 RETRIEVAL-AUGMENTED GENERATION.

For RAG tasks, we validate that APE retains most of the sequential encoding capability while accommodating more and longer contexts, mitigating retrieval errors and outperforming encoding baselines.

5.1.1 RETRIEVAL FOR MULTI-TURN QUESTION ANSWERING.

Setup. APE is evaluated on five conversational QA tasks using ChatRAGBench (Liu et al., 2024b). For each query, 1–100 text chunks are prepared, with retrievers employed to select the top- n most relevant chunks as input. We utilize three different retrievers of varying quality: Contriever (Izacard et al., 2021), GTE-base Li et al. (2023), and Dragon-multiturn Liu et al. (2024b), and retrieve up to the top-5 chunks for evaluation. The base model used is LLAMA3-CHATQA-1.5-8B. To measure performance drop after our alignment steps, the same retrieved texts are used for both APE and sequential encoding.

Results. Table 1 shows that switching from sequential encoding to APE results in performance drops of 0.51%, 0.92%, and 1.14% across different retrievers, respectively. Notably, while this drop increases with retriever quality, APE still keeps 97% of the sequential encoding performance for the best retriever. Moreover, APE can directly input all texts without retrieval process, achieving superior performance.

5.1.2 RETRIEVAL FOR LONG-CONTEXT UNDERSTANDING.

Setup. Our evaluation involves eight tasks on LongBench (Bai et al., 2023). Given the long context, we split it into chunks with a default size of M words, employ Contriever (Izacard et al., 2021) to compute the embeddings of the text chunks and the query, and retrieve the top- N chunks according to the cosine similarity of their embeddings to the query embedding. M and N vary across different

Table 1: Comparison between APE and sequential encoding using three retrievers on ChatRAG-Bench.

Method	INSCIT	Doc2Dial	TopicCQA	Qrecc	QuAC	Average
Contriever, Sequential	19.97	23.85	30.49	46.75	26.57	29.53
Contriever, APE	19.88	23.28	28.84	46.28	26.80	29.02
Δ	-0.09	-0.57	-1.65	-0.47	+0.23	-0.51
GTE-base, Sequential	21.58	32.35	33.41	46.54	30.69	32.91
GTE-base, APE	20.85	30.99	31.92	45.83	30.35	31.99
Δ	-0.73	-1.36	-1.49	-0.71	-0.34	-0.92
Dragon-multiturn, Sequential	25.42	36.27	36.10	49.01	35.12	36.38
Dragon-multiturn, APE	23.84	34.93	33.80	48.70	34.92	35.24
Δ	-1.58	-1.34	-2.30	-0.31	-0.20	-1.14
All texts, APE	27.22	36.13	35.72	49.15	35.70	36.78

Table 2: Comparison between APE and baselines on LongBench across different models using RAG. C denotes Contriever, and $M \times N$ indicates retrieval of the top- N chunks, each containing M words.

Model	MuSiQue	Qasper	2WikiMQA	DuRead	HotpotQA	NarratQA	MFQA_zh	MFQA_en	Avg.
LLAMA-3-8B-INSTRUCT	20.70	41.05	30.02	9.55	45.90	20.98	58.54	45.04	33.97
C200x20, Sequential	27.93	42.71	38.35	12.65	49.60	22.78	57.82	48.94	37.60
C4000x20, PCW	18.82	42.59	40.99	21.57	47.09	23.29	54.40	45.05	36.73
C4000x20, APE	26.19	42.32	44.43	23.13	49.71	30.71	55.03	45.41	39.62
MISTRAL-7B-INSTRUCT-V0.3	10.05	31.08	22.12	17.68	32.09	19.68	32.03	40.38	25.64
C200x20, Sequential	11.58	21.98	24.44	20.80	32.79	16.06	34.43	38.40	25.06
C4000x20, PCW	17.58	35.57	32.97	18.70	37.05	14.10	34.69	40.14	28.85
C4000x20, APE	20.30	36.81	34.37	21.89	42.33	20.49	40.20	44.03	32.55
GEMMA-2-9B-IT	22.57	39.99	48.06	27.40	47.49	23.11	50.81	45.35	38.10
C200x10, Sequential	30.69	42.86	53.55	28.04	52.05	24.45	50.25	48.34	41.28
C2000x20, PCW	26.27	46.69	47.59	23.43	48.95	27.11	56.69	49.81	40.82
C2000x20, APE	33.38	47.72	49.49	28.43	56.62	30.41	56.52	50.84	44.18
LLAMA-3.1-8B-INSTRUCT	22.18	46.81	40.58	34.61	43.97	23.08	61.60	51.89	38.98
128K, Sequential	28.35	47.20	40.81	33.34	53.46	30.57	61.97	53.25	42.24
C200x20, Sequential	30.62	42.33	44.39	33.51	49.97	23.87	56.87	55.14	40.22
C4000x20, PCW	21.23	41.52	44.87	31.11	49.47	19.98	60.90	51.19	38.44
C4000x20, APE	26.88	43.03	50.11	32.10	55.41	30.50	62.02	52.51	42.86

encoding methods. We compare with sequential encoding with and without RAG, and PCW, using LLAMA-3-8B-INSTRUCT (Dubey et al., 2024), MISTRAL-7B-INSTRUCT-V0.3 (Jiang et al., 2023), GEMMA-2-9B-IT (Team et al., 2024), and LLAMA-3.1-8B-INSTRUCT as base models.

Results. In Table 2, APE consistently improves performance across all models, achieving a 5.6% average gain over base models and outperforming sequential RAG baselines by 3.3% through retrieval of more and longer contexts. The superior performance over PCW further showcases the effectiveness of our alignments. Notably, APE surpasses the 128K-context variant of LLAMA-3.1-8B-INSTRUCT by placing retrieved texts within the 8K context window, mitigating the "lost in the middle" phenomenon.

5.2 IN-CONTEXT LEARNING

Setup. We evaluate APE on three ICL tasks using the LM Evaluation Harness (Gao et al., 2024):: GSM8K (8-shot) (Cobbe et al., 2021), TriviaQA (5-shot) (Joshi et al., 2017), and MMLU (5-shot) (Hendrycks et al., 2020). Experiments use the same base models as in our LongBench evaluations.

Baselines. We compare parallel encoding (PCW) to show the improvement from APE’s alignment steps. Sequential encoding with varying numbers of shots (i.e., 1-shot, half-shots, and full-shots) are also employed to measure the gap from the ideal scenarios. All methods can access all input examples.

Results. In Figure 8, APE significantly surpasses parallel encoding with average improvements of 15.4% on GSM8K, 4.7% on TriviaQA, and 3.5% on MMLU. When compared with the 1-shot sequential baseline with comparable context length, our method consistently yields superior results. Moreover, APE achieves better performance than half-shot sequential encoding in 8/12 settings and preserve 93% accuracy comparing to the full-shot sequential encoding. Additionally, our results suggest that the LLAMA family exhibits enhanced compatibility with parallel encoding, potentially due to the stronger directional alignment of initial tokens from different contexts (see Figure 3a). Across different tasks, the performance gap between APE and full-shot sequential encoding is the largest on GSM8K, suggesting that while APE maintains most capabilities, its effectiveness may decrease as task complexity increases.

5.3 MANY-SHOT CONTEXT-AUGMENTED GENERATION

Setup. To demonstrate the scalability of APE, we employ four RAG and ICL tasks from the LOFT benchmark (Lee et al., 2024), each involving hundreds of texts that provide additional information. We

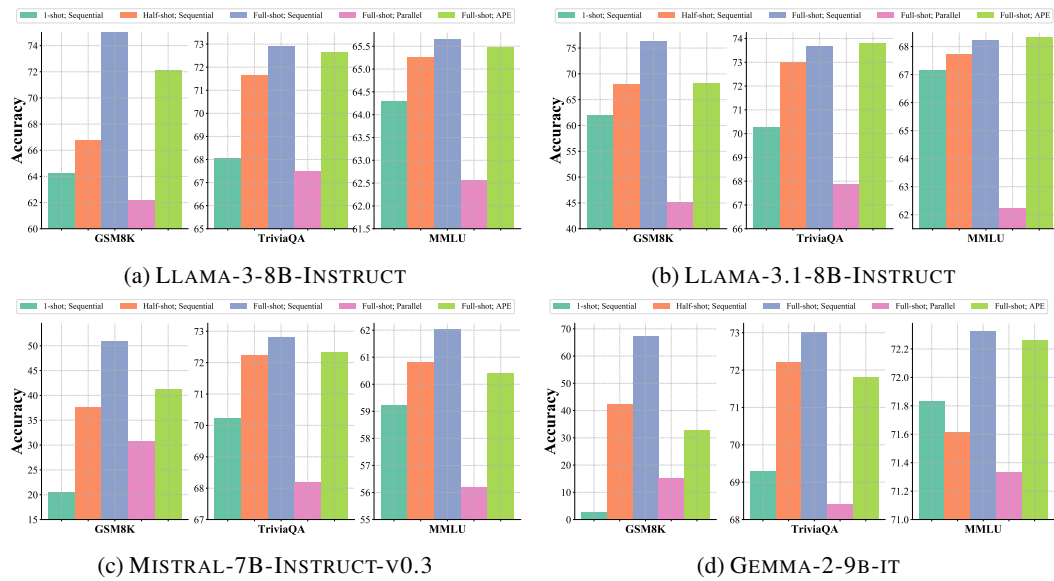


Figure 8: Performance comparison of APE, parallel encoding, and sequential encoding on ICL tasks.
 Table 3: Comparison between APE and sequential encoding in various many-shot RAG and ICL tasks.

Method	Retrieval-augmented Generation				In-context Learning			
	ArguAna	FEVER	NQ	SciFact	Date	Salient	Tracking7	Web
Sequential, Zero-shot	11.15	7.78	17.78	7.74	20.00	8.89	1.12	8.89
Sequential, Few-shot	11.20	9.78	17.81	9.49	36.64	38.89	6.67	38.89
Sequential, Half-shot	15.34	13.12	19.64	16.12	45.55	42.22	8.89	55.56
Sequential, Full-shot	12.84	14.19	24.54	16.88	46.67	46.67	8.89	58.89
APE, Full-shot	16.32	14.70	21.91	15.72	43.33	45.55	8.89	58.89

employ LLAMA-3.1-8B-INSTRUCT as our base model to compare APE with sequential encoding, both applied to the same many-shot long-context inputs. The total context lengths for RAG and ICL tasks are 128K and 32K, respectively. We also include the zero-shot, few-shot (≤ 5), and half-shot variants of sequential encoding as baselines. For metrics, F1 score and EM are used in RAG and ICL.

Results. As shown in Table 3, APE achieves performance comparable to sequential encoding when processing the same many-shot long-context inputs, showcasing its ability to efficiently encode hundreds of texts in parallel. Notably, for RAG tasks, it outperforms sequential encoding on ArguAna and FEVER. While APE is expected to reduce performance, it recovers this drop by positioning all texts close to the query. This mechanism addresses the "lost in the middle" problem in long-context LLMs. Additionally, for ICL tasks, APE can learn from examples as effective as sequential encoding.

5.4 EFFICIENCY EVALUATION

Setup. We measure the latency for sequential encoding, MInference (Jiang et al., 2024a), and APE on the Llama-3.1-8B-Instruct (Dubey et al., 2024) model. Our evaluation is conducted on an H100 GPU with batch sizes of 1 and 4. The query and generation lengths are fixed at 256 tokens, while the input context lengths ranged from 2K to 128K tokens. We employ VLLM (Kwon et al., 2023) as our inference engine and measure both prefilling time (Time to first token) and total inference time.

Results. Comparing to sequential encoding and MInference, APE can accelerate inference up to $4.5\times$ and $2.2\times$ respectively for long-context scenarios in Table 4. Notably, for 128K-token contexts, APE reduces prefilling time by $28\times$ compared to MInference. APE’s prefilling overhead exhibits linear scaling and consumes less than 10% of inference time, while baselines require over 50% as context length increases. APE also demonstrates superior versatility across various settings, whereas MInference slows inference with additional overhead when processing short contexts and large batches.

Table 4: Latency on H100 GPU: [prefill / total time (ms)]. The gray text in brackets is batch size.

Methods	4K	8K	16K	32K	64K	128K
Sequential Encoding (1)	132/2383	276/2558	635/3204	1646/4713	4892/8804	16635/22252
MInference (4)	964/3221	1297/3609	1623/4201	2206/5274	3563/7458	6457/12085
APE (4)	28/2279	40/2322	68/2637	122/3189	226/4138	421/6038
Sequential Encoding (4)	523/3129	1084/4179	2558/6474	6599/12055	19468/28367	65928/83869
MInference (1)	3789/6373	4995/8099	6300/10244	8470/13944	13462/22313	24199/42013
APE (4)	69/2675	94/3189	150/4066	298/5754	430/9329	858/18799

6 ANALYSIS

In this section, we present analyses to answer the following research questions: **RQ1**: Does each alignment process in APE function effectively? **RQ2**: Can APE improve performance for real-world RAG tasks? **RQ3**: Can APE extend LLM context window size in long-context scenarios without RAG?

6.1 HOW DOES EACH ALIGNMENT PROCESS IN APE CONTRIBUTE TO THE PERFORMANCE?

In Table 5, we conduct an ablation study to examine each alignment process in APE, including the shared prefix (P), attention temperature (T), and scaling factor (S). We present results averaged across the four models evaluated in Figure 8. Our findings indicate that incorporating each of these components can consistently enhance performance across all tasks, with average improvements of 5.19%, 0.59%, and 2.07%, respectively. Among them, adjusting the attention temperature yields minimal performance gains without the complementary effect of the scaling factor.

Table 5: Ablation study of APE alignments on ICL tasks. P : shared prefix, T : attention temperature, S : scaling factor.

P	T	S	GSM8K	TriviaQA	MLLM
			38.25%	67.99%	63.09%
✓			50.42%	70.76%	63.70%
✓	✓		51.15%	71.03%	64.49%
✓	✓	✓	53.62%	72.64%	66.62%

6.2 CAN APE IMPROVE PERFORMANCE FOR REAL-WORLD RAG APPLICATIONS?

In Table 6, we evaluate APE’s performance in real-world RAG scenarios using the CRAG benchmark (Yang et al., 2024). Task 1 augments the model with several webpages, while Task 2 provides an additional knowledge graph as another retrieval source. In our experiments, the sequential encoding baseline is limited to retrieving 4K tokens of context, whereas APE can process 20 parallel segments of 4K tokens each. By incorporating significantly more external data during generation, APE consistently outperforms sequential encoding that has limited context sizes while reducing latency. Moreover, the improvement in Task 2 further shows the effectiveness of APE in merging text from multiple sources.

Table 6: Performance and latency comparison using the LLAMA-3-8B-INSTRUCT model on CRAG.

Task	Model	Latency (ms)	Accuracy (%)	Hallucination	Missing	Score _a
LLM only	LLAMA-3-8B-INSTRUCT	682	22.14	48.97	28.90	-26.83
Task 1	LLAMA-3-8B-INSTRUCT	1140	23.28	29.49	47.22	-6.21
	+APE	1054	25.53	21.30	37.93	-0.41
Task 2	LLAMA-3-8B-INSTRUCT	1830	24.46	28.38	47.15	-3.92
	+APE	1672	27.04	18.74	37.32	2.16

6.3 CAN APE EXTEND CONTEXT LENGTHS IN LONG-CONTEXT SCENARIOS WITHOUT RAG?

Table 7 examines the effectiveness of APE when processing a single long context input for the LLAMA-3-8B-INSTRUCT model on LongBench (Bai et al., 2023). To accommodate the long context within our APE, we split it into multiple segments with of less than 7,500 tokens, appending the final 500 tokens to the query. Our results indicate that APE enhances performance across 10/11 tasks, yielding an average improvement of 6.6% compared to the sequential encoding baseline with limited context window size.

Table 7: Performance comparison across different long-context tasks on LongBench (Bai et al., 2023).

Method	NarratQA	Qasper	MultiFQA	GovReport	QMSum	LCC
LLAMA-3-8B-INSTRUCT	19.32	32.83	43.38	27.89	22.40	53.22
+APE	26.87	39.14	59.12	29.10	23.08	66.09
Method	RepoBench-P	HotpotQA	2WikiMQA	MuSiQue	MultiNews	Average
LLAMA-3-8B-INSTRUCT	38.15	44.24	21.01	20.47	23.63	31.50
+APE	49.43	50.11	28.06	25.79	22.40	38.11

7 CONCLUSION

In summary, the work explores the potential of parallel encoding in CAG, which pre-cache KV states for fast inference and re-use positions for long context but leads to worse performance. To address this, we propose APE, a training-free method that enables accurate, fast, and long CAG. APE achieves this by aligning the attention distribution of parallel encoding with sequential encoding via three steps: shared

prefix, adaptive temperature, and scaling factor. Our method improves efficiency and performance in various RAG and ICL tasks, while successfully scaling to process hundreds of chunks in parallel.

REFERENCES

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Rishabh Agarwal, Avi Singh, Lei M Zhang, Bernd Bohnet, Stephanie Chan, Ankesh Anand, Zaheer Abbas, Azade Nova, John D Co-Reyes, Eric Chu, et al. Many-shot in-context learning. *arXiv preprint arXiv:2404.11018*, 2024.
- Akari Asai, Zexuan Zhong, Danqi Chen, Pang Wei Koh, Luke Zettlemoyer, Hannaneh Hajishirzi, and Wen-tau Yih. Reliable, adaptable, and attributable language models with retrieval. *arXiv preprint arXiv:2403.03187*, 2024.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.
- Harrison Chase. Longchain, 2022. URL <https://github.com/langchain-ai/langchain>.
- Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35: 16344–16359, 2022.
- Jacob Devlin. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. A survey on in-context learning. *arXiv preprint arXiv:2301.00234*, 2022.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvassy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024. URL <https://zenodo.org/records/12608602>.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Aman Gupta, Anup Shirgaonkar, Angels de Luis Balaguer, Bruno Silva, Daniel Holstein, Dawei Li, Jennifer Marsman, Leonardo O Nunes, Mahsa Rouzbahman, Morris Sharp, et al. Rag vs fine-tuning: Pipelines, tradeoffs, and a case study on agriculture. *arXiv preprint arXiv:2401.08406*, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *arXiv preprint arXiv:2407.02490*, 2024a.
- Ziyan Jiang, Xueguang Ma, and Wenhui Chen. Longrag: Enhancing retrieval-augmented generation with long-context llms. *arXiv preprint arXiv:2406.15319*, 2024b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Jinhyuk Lee, Anthony Chen, Zhuyun Dai, Dheeru Dua, Devendra Singh Sachan, Michael Boratko, Yi Luan, Sébastien MR Arnold, Vincent Perot, Siddharth Dalmia, et al. Can long-context language models subsume retrieval, rag, sql, and more? *arXiv preprint arXiv:2406.13121*, 2024.
- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.
- Zhenyu Li, Yike Zhang, Tengyu Pan, Yutao Sun, Zhichao Duan, Junjie Fang, Rong Han, Zixuan Wang, and Jianyong Wang. Focusllm: Scaling llm’s context by parallel decoding. *arXiv preprint arXiv:2408.11745*, 2024.
- Jerry Liu. Llamaindex, 11 2022. URL https://github.com/jerryjliu/llama_index.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a.
- Zihan Liu, Wei Ping, Rajarshi Roy, Peng Xu, Chankyu Lee, Mohammad Shoeybi, and Bryan Catanzaro. Chatqa: Surpassing gpt-4 on conversational qa and rag. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.
- Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. Parallel context windows for large language models. *arXiv preprint arXiv:2212.10947*, 2022.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- East Sun, Yan Wang, and Lan Tian. Block-attention for efficient rag. 2024. URL <https://api.semanticscholar.org/CorpusID:272832445>.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhu-patiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. Crag-comprehensive rag benchmark. *arXiv preprint arXiv:2406.04744*, 2024.

Howard Yen, Tianyu Gao, and Danqi Chen. Long-context language modeling with parallel context encoding. *arXiv preprint arXiv:2402.16617*, 2024.

André Zayarni, Andrey Vasnetsov, et al. Qdrant, 2024. URL <https://qdrant.tech/>.

A DETAILED EXPERIMENTAL SETUPS FOR SECTION 3.1